

```

%MAE 423 - Project Code
%Humza Khan (50363412)
clc; clear all; close all;

%% project starts here
%inputs
g = 9.81; %m/s
payload = [5, 10, 20]; %kg
h_max = [10000; 20000; 30000] / 3.281; %altitude converted to meters
a_max = [5, 10, 20]; %normalized acc
all_combos = [payload(1), h_max(2), a_max(2);
              payload(3), h_max(2), a_max(2);
              payload(2), h_max(1), a_max(2);
              payload(2), h_max(3), a_max(2);
              payload(2), h_max(2), a_max(1);
              payload(2), h_max(2), a_max(3);
              payload(2), h_max(2), a_max(2)]; %all possible inputs

SM = 2; %static margin
rho_s = 2700; %kg/m^3
rho_p = 1772; %kg/m^3
sigma_s = 150e6; %MPa ---> Pa
N = 4; %Number of Fins
R_opt = zeros(7,1); %initializing variable
W_eq = zeros(7,1); %initializing variable
t_burn = zeros(7,1); %initializing variable
mach = zeros(7,1); %initializing variable
[T_sl,a_sl,P_sl,rho_sl] = atmosisa(0);
T_stp = 298.15; %K Temp at STP
gamma = 1.4; %assuming diatomic atmosphere
R = 287; %universal gas constant
a_0 = sqrt(gamma.*T_stp.*R);

%=====
%% Solving for R_opt, W_eq, t_b, and Mach Numbers for each case
for i = 1:7
    R_opt(i) = 1 + all_combos(i,3);
    W_eq(i) = sqrt((all_combos(i,2).*g)./(0.5.*log(R_opt(i)).*(log(R_opt(i))-2)+
    ((R_opt(i) - 1)./ R_opt(i))));
    t_burn(i) = ((R_opt(i) - 1)*W_eq(i))/(g*R_opt(i)); %optimal burn time eqn
    mach(i) = W_eq(i)/a_0; %solving for mach numbers @ SL
end
%=====
%% solving for gas dynamics stuff incl. total temp and pressure, mass, cg, cp
T_1 = T_stp.*(1+0.5.*(gamma-1).*mach.^2) ; %K total temp
P_0 = P_sl.*(T_1./T_stp).^(gamma/(gamma-1)) ; %Pa total pressure

guess_d = linspace(0,1,5000); %guess D range
guess_L = linspace(0,5,5000); %guess L range

```

```

%=====
%starting the loop
for k = 1:7
    delta = guess_d .* P_0(k,1) ./ (2 * sigma_s);
    L_D = guess_d + guess_L';
%=====
    %mass calculations
    %fin
    fin_sa = 0.5 .* guess_d .* guess_d; %SA of fin
    m_fin = N .* fin_sa .* delta .* rho_s; %mass of 4 fins
    m_fin_matrix = repmat(m_fin,[5000,1]); %matrix sizing
%=====
    %cone
    cone_sa = pi.*(0.5.*guess_d).*(sqrt( (0.5.*guess_d).^2 + guess_d.^2)); %SA of cone (no bottom circle)
    m_cone = cone_sa .* delta .* rho_s; %mass of the cone
    m_cone_matrix = repmat(m_cone,[5000,1]); %matrix sizing
%=====
    %cylinder
    cylinder_sa = 2.*pi.*(L_D).*(0.5.*guess_d); %SA of cylinder (no top/bottom circle)
    m_cylinder = cylinder_sa .* delta .* rho_s; %no need to adjust sizing as dependant on both L and D.
%=====
    %full structure
    m_structure = m_fin_matrix + m_cone_matrix + m_cylinder;
%=====
    %propellant mass
    m_p = (R_opt(k,1)-1) .* (m_structure + all_combos(k,1));
%=====
    %propellant length
    L_p = m_p ./ (guess_d.^2 .* (pi.*rho_p/4));
%=====
    %X.CG Calculation
    X.CG_nose = (2./3).*guess_d;
    X.CG_cylinder = guess_d + 0.5.*(L_D);
    X.CG_fins = (2./3).*guess_d + (L_D);
    X.CG_prop = (0.5.*L_p) + ((2.*guess_d + guess_L') - L_p);
    m_total = m_structure + m_p + all_combos(k,1);
    X.CG = ((X.CG_fins.*m_fin_matrix) + (X.CG_cylinder.*m_cylinder) + (X.CG_prop.*m_p) + (X.CG_nose.*m_cone_matrix) + (X.CG_nose .* all_combos(k,1)))./m_total ;
%=====
    % X_CP calculation - Most Values were obtained from the Centuri Document
    X_conical = (2./3).*guess_d; %recalculating for sanity %center for normal force
    X_conical_matrix = repmat(X_conical,[5000,1]);
    a = guess_d;
    b = 0;
    S = guess_d;
    weird_l_length = sqrt(guess_d.^2 + (0.5.*guess_d).^2);

```

```

CNaf = (4.*N.*(S./guess_d).^2)./(1 + sqrt(1 + (2.*weird_l_length./(a + b)).^2)); %found this to be constant throughout
R = 0.5.*guess_d; %radius
kfb = 1 + (R./(S+R)); %interference factor for 4 fins
CNafb = kfb.*CNaf; %normal force on fins in presence of the body
CNan = 2; %normal force on the nose
x_f = guess_d + guess_L';
m = guess_d;
del_xf = (((m.*(a + 2.*b))./(3.*(a + b)))) + (1/6).*(a + b - (a.*b)./(a+b));
xf_bar = x_f + repmat(del_xf,[5000,1]); %total center of pressure in inches
CNa = CNafb + CNan; %total norm coeff
%solving for x_cp
X_CP = (CNan.*X_conical_matrix + repmat(CNafb,[5000,1]).*xf_bar)./CNa; %center of pressure location from the top of the rocket
%=====

lamda = all_combos(k,1)./(m_structure+m_p);

%stability analysis
const_1 = abs((X_CP - X_CG) ./ repmat(guess_d,[5000,1]));
const_1norm = abs(const_1) - 2; %enforces SM normalizes
const_1norm1 = abs(const_1norm);

min_const1(1,k) = min(const_1norm1(:));
[min_row, min_col] = find(const_1norm1 == min_const1(1,k));

const_2 = lamda;

max_lamda(1,k) = const_2(min_row, min_col);
[max_row, max_col] = find(const_2 == max_lamda(1,k));

%=====
%% Extracting Case-By-Case Values in this section
P0_fraction(1,k) = P_0(k,1)/P_sl;
dD_fraction(1,k) = delta(1,max_col)/guess_d(1,max_col);
final_d(1,k) = guess_d(1,max_col);
final_L(1,k) = guess_L(1,max_row);
LD_fraction(1,k) = final_L(1,k)/final_d(1,k);
final_XCG(1,k) = X_CG(max_row,max_col);
final_XCP(1,k) = X_CP(max_row,max_col);
final_mp(1,k) = m_p(max_row,max_col);
final_ms(1,k) = m_structure(max_row,max_col);
final_m0(1,k) = m_total(max_row,max_col);
final_lamda(1,k) = max_lamda(1,k);
epsilon(1,k) = m_structure(max_row,max_col)/(m_structure(max_row,max_col) + m_p %
(max_row,max_col));
%=====

```

end

```
%thrust = m_p.*W_eq./t_burn; ----> Thrust eqn used.
```

```
%=====
```

```
%% organizing data in arrays
```

```
Cases = all_combos;
```

```
dataset_extract = [R_opt, W_eq, t_burn, P0_fraction', dD_fraction',final_d',↵  
final_L', LD_fraction', final_XCG', final_XCP', final_mp', final_ms', final_m0',↵  
final_lamda',epsilon'];
```

```
FINAL_TABLE = [Cases, dataset_extract]; %final answer that was used to form the↵  
table.
```

```
%=====
```